ClaimSense AI capstone

Brief, submissions and grading

1. Overview

Participants act as the forward deployed engineer on a five-week client engagement with a motor insurer. The engagement runs from discovery to handover and is assessed once, on the final submission.

The client requires an adjuster's first pass on a claim to be completed in minutes rather than hours, with the reasoning recorded and a licensed adjuster accountable for every outcome.

2. The problem

Adjusters process every claim manually across three systems: claims management, policy administration and a document store. A single adjuster gathers the file, checks coverage, assesses fraud, total loss and injury exposure, and writes the first decision. Decisions vary between adjusters, the reasoning is not recorded, and fraud referral is triggered by a score threshold rather than a reading of the file.

Extending the existing rule table does not address this. Coverage depends on the wording of a policy clause, and fraud assessment depends on the consistency of a narrative across documents.

3. Scope of work

The system under construction is a claims adjudication co-pilot. Given a claim, it returns a first decision, the reasoning behind that decision, and the evidence the reasoning rests on. For a user not entitled to the claim, it refuses the request and returns no claim data. A licensed adjuster confirms every outcome.

Three constraints are fixed and are not subject to participant design:

1. **Deterministic operations are implemented in code.** Identity, access control, policy dates and monetary calculations are never delegated to a model.
2. **Judgment is delegated to a model, and is cited.** Every judgment states the evidence it relied on.
3. **Synthetic data only.** No production or personally identifiable data is introduced at any stage.

All remaining design decisions rest with the participant and are assessed on the reasoning presented.

4. Technology standard

Capability required	Standard for this cohort
Typed, verifiable model output	OpenAI Agents SDK
Retrieval with source attribution	A searchable policy clause corpus
Controlled access to client systems	A participant-owned service with authentication
Portable execution	Containers
Change control	CI with an evaluation gate

Capability required	Standard for this cohort
Hosting, observability and recovery	AWS

Runtime language is Python; the web framework is at the participant's discretion. Participants require an AWS account, an OpenAI API key, Docker, Python 3.12 or later, and Node 20 or later.

5. Milestones

One milestone per engagement week. All deliverables are committed to the participant's capstone repository. Weekly deliverables receive written feedback and are not graded individually.

Milestone 1 — Discovery and scoping (Week 1)

Establish what the client actually needs and what success will be measured against.

1. Engagement charter of one to two pages, naming the three client systems and their integration points.
2. Stakeholder map and RACI, identifying the approver, the reviewer of co-pilot output, the daily user, and who can block rollout.
3. Four quantified success criteria covering decision accuracy, pipeline latency, adjuster confirmation rate and fraud routing accuracy.
4. `DECISIONS.md`, opened with at least three dated architectural decisions and the reason for each.
5. An initial set of 20 evaluation cases spanning all four decision outcomes, including adversarial cases.

Milestone 2 — Statement of work, pricing and architecture (Week 2)

Convert the discovery findings into a contract, a price and a design.

1. Statement of work of two pages: scope, acceptance criteria, intellectual property and data handling terms, and the five-week timeline.
2. Architecture diagram with every data flow labelled and the boundary between deterministic code and model judgment marked explicitly.
3. Pricing model with the underlying assumptions stated.
4. Risk register of five risks with likelihood, impact and mitigation.
5. Typed input and output contracts for all three client systems, including idempotency rules for any write operation.
6. Synthetic data shapes for claims, policies and documents.
7. Cloud account provisioned, with the created resources evidenced.

Milestone 3 — Production agents (Week 3)

Deliver a working pipeline under change control.

1. Agent set implemented on the OpenAI Agents SDK with strict typed outputs.
2. Container build and local orchestration; the stack starts with a single command.
3. Retrieval over the policy corpus, with coverage output citing the clause identifiers actually retrieved.
4. CI pipeline in which a failing evaluation gate blocks the image push.
5. All four decision outcomes produced correctly, with model-generated verdicts distinguishable from deterministic fallback output.

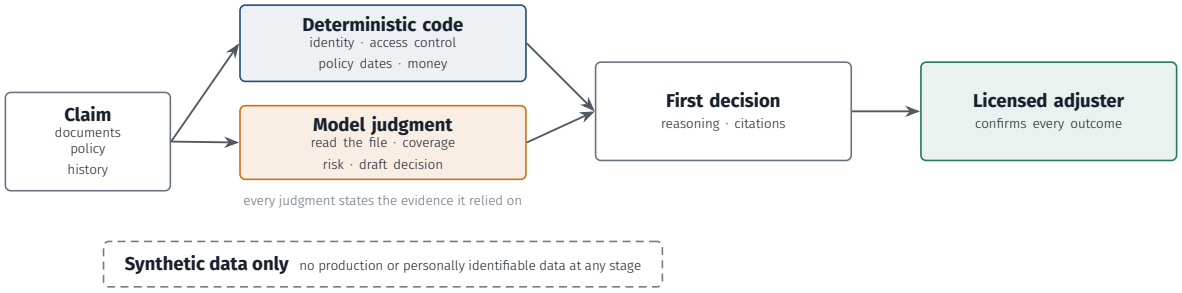


Figure 1: Division of responsibility across the system

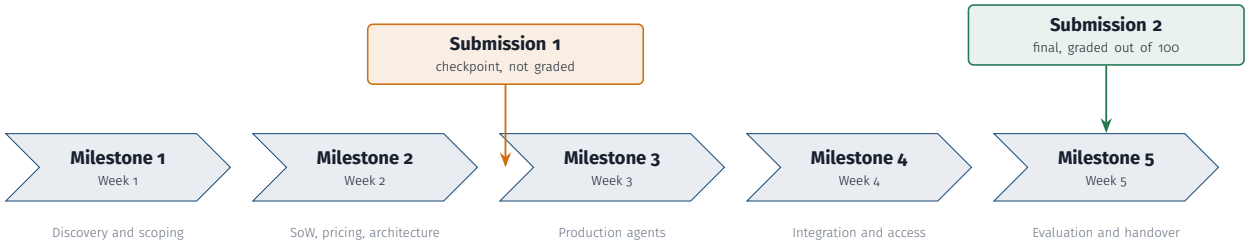


Figure 2: The five milestones and the two submission points

Milestone 4 — Integration and access control (Week 4)

Establish the trust boundary and the audit trail.

1. Data access boundary deployed and callable.
2. Authorisation enforced on every route; requests without a valid token are refused.
3. Access control implemented in code and executed before any agent is invoked.
4. Two-user access test recorded, with both requests and both responses, on the same claim.
5. An audit record written for every call, granted or denied, carrying user, action and resource.

Milestone 5 — Evaluation, observability, reliability and handover (Week 5)

Demonstrate that the system is safe to operate, and transfer it.

1. Evaluation suite of 150 cases including adversarial families: prompt injection, cross-record leakage and forced auto-decision. The gate passes with the model required.
2. Operational dashboard reporting error rate, call volume and p95 latency, with one alarm defined.
3. Incident drill: a deliberate regression, detection, rollback, and a one-page incident report recording time to verified recovery.
4. Runbook covering four procedures: add a user, roll back a deployment, escalate a flagged claim manually, and update the policy corpus.
5. Client presentation with a live demonstration.
6. Handover pack consolidating all engagement artefacts.

6. Acceptance criteria for the delivered system

The delivered system must satisfy all five criteria below, each demonstrable by a first-time user working alone from the deployed system and its documentation.

1. **Decision on any claim.** Any claim in the corpus returns a first decision, the reasoning, and citations to the evidence relied on.
2. **Refusal on entitlement.** A request from a user not entitled to the claim returns no claim data, and the refusal appears in the audit trail.

3. **Attribution within a response.** Model judgment is distinguishable from deterministic computation in every response.
4. **Reproducible evidence.** The evaluation suite, including cases constructed to make the system fail, runs on the submitted commit and reports its results.
5. **Operability without the author.** The system can be operated from the runbook, observed on the dashboard, and recovered after an induced failure.

7. Submissions

Two submissions are made through the course's Google Forms.

Submission 1 — Checkpoint (due before the Week 3 session)

Confirms environment readiness and progress against Milestones 1 and 2. Not graded.

Required contents:

1. Capstone repository URL.
2. Declaration of which Milestone 1 and Milestone 2 deliverables are complete.
3. Evidence that the stack runs under the participant's own account: one screenshot of the running service or the provisioned resources.
4. A statement of outstanding blockers, including any account, credential or quota limitations.

Submission is required regardless of completeness.

Submission 2 — Final (due at the end of Week 5)

Graded out of 100.

Required contents:

1. Codebase at the `final` tag, containing the deliverables of all five milestones.
2. Deployed frontend URL, reachable until grading concludes.
3. Observation document covering six sections: deployment, evaluation, observability, platform reliability, handover, and observations.
4. Demonstration recording of no more than ten minutes.
5. CI run on the submitted commit.
6. Annotated screenshot set evidencing the AWS deployment: the running console, CloudWatch dashboards and alarms, the container registry, log and audit records, and the CI run. Each screenshot is annotated to identify what it shows.

8. Assessment

A single grade out of 100 is awarded on Submission 2. Weekly deliverables receive written feedback and are not graded individually.

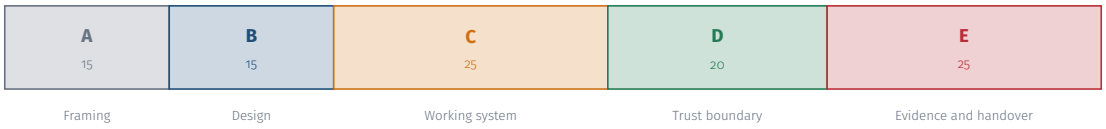


Figure 3: Weighting of the five assessed areas, out of 100

Each area is assessed on demonstrated outcomes, not on the presence of named artefacts. Three bands apply in every area.

Area	Weight	Strong (80 to 100%)	Partial (40 to 79%)	Weak (0 to 39%)
A. Framing the problem	15	The bottleneck identified is the one that governs adjuster throughput, and the success criteria are quantified closely enough to settle a later dispute about whether the engagement worked	The problem is restated as the client described it, or the success criteria are stated without numbers	No charter, or success criteria absent
B. Design under constraint	15	The boundary between deterministic code and model judgment is drawn explicitly and justified, the discarded alternatives are recorded, and the risk register anticipates failures that later occurred	The design is documented but the reasoning is absent, or the code and model boundary is left implicit	No architecture, or no risk register
C. The working system	25	The system starts from a clean checkout, runs end to end, cites retrieved clauses in coverage output, and returns correct outcomes on claims it was not tuned against	A stage is stubbed, citations are not grounded in retrieved sources, or the change control gate does not block a failing build	No end to end run, or outcomes originate from the deterministic fallback only
D. The trust boundary	20	Authorisation is enforced in code ahead of any agent, holds on every route, and every call is recorded with user, action and resource, including refusals	One route is unguarded, or the two-user test is not evidenced	No boundary, or access control expressed as a model instruction
E. Evidence and handover	25	The evaluation suite includes adversarial families and runs on the submitted commit, operational failure is detected and recovered under drill conditions, and the runbook is sufficient for a first-time operator	The suite omits an adversarial family, the dashboard carries no alarm, or the runbook omits a procedure	The suite does not run, or no runbook is delivered

Across all five areas, **Strong** requires that the outcome be demonstrated on evidence produced by the participant and verifiable independently. **Partial** applies where an outcome is asserted but not demonstrated, or holds only under expected conditions. **Weak** applies where the outcome is not achieved or the evidence contradicts the claim.